

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1460

**PRIMJENA SPECIFIKACIJE OPENAPI U PROCESIMA
RAZVOJA I DOKUMENTIRANJA HTTP API-JA**

Adrian Brnić

Zagreb, lipanj 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1460

**PRIMJENA SPECIFIKACIJE OPENAPI U PROCESIMA
RAZVOJA I DOKUMENTIRANJA HTTP API-JA**

Adrian Brnić

Zagreb, lipanj 2026.

DIPLOMSKI ZADATAK br. 1460

Pristupnik: **Adrian Brnić (0036531884)**
Studij: Računarstvo
Profil: Programsko inženjerstvo i informacijski sustavi
Mentor: doc. dr. sc. Slaven Zakošek

Zadatak: **Primjena specifikacije OpenAPI u procesima razvoja i dokumentiranja HTTP API-ja**

Opis zadatka:

Kvalitetna dokumentacija predstavlja jedan od najvažnijih elemenata svakog HTTP API-ja, a njezina je primarna svrha ubrzati razvoj i pojednostaviti održavanje softvera te olakšati integraciju sustava. Specifikacija OpenAPI definira tehnološki neutralan format opisa koji programerima omogućuju da bez uvida u izvorni kôd prepoznaju dostupne krajnje točke, strukture podataka i formate zahtjeva i odgovora, a softverskim alatima omogućuje generiranje interaktivne API dokumentacije, klijentskog i poslužiteljskog koda, automatiziranih testova i mehanizama validacije. Identificirati i proučiti relevantne alate, po mogućnosti otvorenog koda, za uređivanje i vizualizaciju OpenAPI dokumenata, te generiranje klijentskog i poslužiteljskog koda, uključujući kod za testiranje i validaciju. Istražiti i objasniti dva temeljna pristupa razvoju HTTP API-ja: pristup temeljen na specifikaciji (engl. specification-first) i pristup temeljen na kodu (engl. code-first). Za oba pristupa opisati tipični tijek razvoja i analizirati njihove prednosti i nedostatke. Na temelju ove analize predložiti praktične kriterije za odabir prikladnog pristupa u različitim scenarijima. Također je potrebno opisati važnost verzioniranja API-ja i definiranja sigurnosnih shema. Na odabranom oglednom sustavu demonstrirati praktičnu primjenu specifikacije OpenAPI. Odabrani testni sustav treba biti jasno opisan, a proces njegove izgradnje dokumentiran i kritički ocijenjen.

Rok za predaju rada: 26. lipnja 2026.

Sažetak

Primjena specifikacije OpenAPI u procesima razvoja i dokumentiranja HTTP API-ja

U ovom se radu opisuju mogućnosti OpenAPI specifikacije u razvoju HTTP API-ja i generiranju dokumentacije. Predstavljeni su povijest i motivacija za razvoj specifikacije, struktura OpenAPI dokumenta, alati za rad sa specifikacijom te pristupi razvoju API-ja. U sklopu rada razvijen je ogledni sustav korištenjem dva različita pristupa razvoju API-ja koja nudi OpenAPI specifikacija – pristupa temeljenog na kodu te pristupa temeljenog na specifikaciji. Na temelju iskustva i znanja stečenih izradom oglednog sustava predloženi su kriteriji usporedbe dvaju različitih pristupa. Osim toga, predstavljeni su načini upravljanja verzijama API-ja te sigurnosne sheme koje specifikacija podržava.

Ključne riječi: HTTP API, dokumentacija, OpenAPI specifikacija, pristup temeljen na kodu, pristup temeljen na specifikaciji

Summary

Application of the OpenAPI specification in HTTP API development and documentation processes

This paper covers the capabilities of the OpenAPI specification in HTTP API development and documentation generation. Presented are the history and motivation behind the development of the specification, the structure of an OpenAPI document, tools for working with the specification and the approaches to API development. As part of the paper, a demonstration system was developed using two different approaches provided by the OpenAPI specification – the code-first approach and the specification-first approach. Based on the findings during the development of the demonstration system, criteria for comparing the two different approaches were defined. In addition, methods of API versioning and the security schemes that can be used are presented.

Keywords: HTTP API, documentation, OpenAPI specification, code-first approach, specification-first approach

Sadržaj

Sažetak.....	1
Summary.....	2
Sadržaj	3
Uvod	6
1. HTTP API i dokumentacija	7
1.1. HTTP API.....	7
1.2. Dokumentacija API-ja	8
2. OpenAPI specifikacija	11
2.1. Swagger	11
2.2. Preimenovanje u OpenAPI	12
2.3. Struktura OpenAPI dokumenta	12
2.3.1. Opće informacije (info)	13
2.3.2. Poslužitelji (servers)	13
2.3.3. Grupiranja (tags).....	14
2.3.4. Putanje (paths)	15
2.3.5. Komponente (components).....	17
2.3.6. Sigurnost (security)	20
3. Alati za rad s OpenAPI specifikacijom	21
3.1. Uređivanje koda specifikacije	21
3.1.1. Swagger Editor	21
3.1.2. Stoplight Studio	21
3.2. Vizualizacija API-ja	22
3.2.1. Swagger UI.....	22
3.2.2. Redoc	23

3.3.	Generiranje koda.....	23
3.3.1.	Swagger Codegen	23
3.3.2.	OpenAPI Generator	24
3.4.	Validacija.....	25
3.4.1.	Spectral	25
3.5.	Testiranje	25
3.5.1.	Dredd	25
4.	Ogledni sustav	27
4.1.	Opis sustava.....	27
4.2.	Arhitektura sustava i tehnologije.....	27
4.2.1.	Baza podataka.....	27
4.2.2.	Poslužiteljski dio aplikacije	28
4.2.3.	Klijentski dio aplikacije.....	28
4.3.	Model podataka	30
4.3.1.	Korisnik	30
4.3.2.	Proizvod.....	31
4.3.3.	Narudžba.....	32
4.3.4.	Stavka narudžbe.....	33
5.	Pristupi razvoju API-ja	35
5.1.	Pristup temeljen na kodu	35
5.2.	Pristup temeljen na specifikaciji.....	37
5.3.	Hibridni pristup	39
5.4.	Usporedba pristupa i kriteriji odabira.....	40
6.	Verzioriranje i sigurnost	42
6.1.	Verzioriranje	42
6.1.1.	Upravljanje verzijama pomoću URL-a.....	42

6.1.2.	Upravljanje verzijama pomoću parametra zahtjeva	43
6.1.3.	Upravljanje verzijama pomoću HTTP zaglavlja	44
6.2.	Sigurnost.....	44
6.2.1.	HTTP autentifikacija	45
6.2.2.	Autentifikacija putem API ključa	45
6.2.3.	OAuth2	45
6.2.4.	OpenID Connect.....	46
6.2.5.	Uzajamni TLS	46
Zaključak		48
Literatura		49
Izjava o korištenju umjetne inteligencije.....		51

Uvod

Kvalitetna dokumentacija HTTP API-ja ključan je faktor u razvoju i održavanju modernog softvera. S lošom, nepotpunom ili netočnom dokumentacijom integracija komponenti sustava postaje teža, sporija te zahtijeva više sredstava, a softver postaje podložan greškama i težak za održavanje.

OpenAPI specifikacija je nastala kao odgovor na taj problem - precizno definira krajnje točke sustava, formate zahtjeva i odgovora te podatke koje te krajnje točke primaju. Dokumentacija koja se generira korištenjem OpenAPI specifikacije programeru omogućuje brz i jednostavan uvid u arhitekturu sustava, ali još bitnije, omogućuje strojno čitljiv opis API-ja. Tako se korištenjem OpenAPI specifikacije može generirati kod u izabranom jeziku ili razvojnom okviru, što se naziva pristup temeljen na specifikaciji. Programer može definirati krajnje točke, format zahtjeva i odgovora, ograničenja podataka te tip autentifikacije u datoteci koja se naziva OpenAPI specifikacija ili dokument. Iz te datoteke generiraju se programski kod te dokumentacija.

Suprotno pristupu temeljenom na specifikaciji, OpenAPI specifikacija omogućuje korištenje pristupa temeljenog na kodu. Kod tog pristupa programer generira OpenAPI dokument i dokumentaciju tek nakon što se napiše kod. U osnovi je proces sličan, ali se na ovaj način programeru omogućava fleksibilnost ovisno o vrsti projekta na kojem radi.

Cilj ovog rada je istražiti, primijeniti i demonstrirati različite mogućnosti ove specifikacije i usporediti navedene pristupe u razvoju softvera. Radi lakše demonstracije implementirana je web-aplikacija u *e-commerce* domeni, odnosno web-trgovina. Sustav pokriva upravljanje korisnicima, proizvodima i narudžbama, s JWT autentifikacijom i različitim verzijama sustava kako bi se pokazale mogućnosti OpenAPI specifikacije. Rad je sastavljen od teorijskog dijela, u kojem će se analizirati sama specifikacija i njezine mogućnosti, te praktičnog dijela u kojem će se te mogućnosti demonstrirati na primjerima.

1. HTTP API i dokumentacija

1.1. HTTP API

HTTP API (aplikacijsko programsko sučelje, engl. *Application Program Interface*) je skup logički povezanih komunikacijskih točaka koji pruža uslugu [1] korištenjem HTTP protokola te je danas jedan od najčešćih oblika komunikacije korisnika internetskog preglednika s poslužiteljem na kojem se stranica nalazi. Takav tip komunikacije naziva se klijent-poslužitelj model (engl. *model client-server*, MCS). Ovaj arhitekturni obrazac sastoji se od računala klijenta i računala poslužitelja te mu je temeljno načelo raspodjela njihovih uloga. Računalo klijenta zaslužno je za početak komunikacije te šalje zahtjev, dok računalo poslužitelja na zahtjev šalje odgovor koji može biti uspješan ili neuspješan, ovisno o ispravnosti zahtjeva ili dostupnosti traženih resursa na poslužitelju. MCS model koristi protokole na više slojeva, a najčešće korišteni protokoli su HTTP protokol na aplikacijskom sloju, TCP i UDP protokoli na tranzitnom sloju te IP protokol na internetskom sloju. Ovakva raspodjela odgovornosti rasterećuje računalo poslužitelja i omogućava skaliranje sustava, a istovremeno korisniku pruža bolju sigurnost. MCS model ima svoje nedostatke, a glavni je ovisnost klijenta o poslužitelju, što znači da ako računalo poslužitelja iz bilo kojeg razloga nije dostupno, korisnik neće moći uspostaviti komunikaciju.

Povijesno gledano, komunikacija preko interneta bila je puno jednostavnija te su se u početku koristili protokoli kao što su Telnet, FTP ili SMTP. Osim njih, koristila se komunikacija preko utičnica (engl. *socket*) te kasnije preko SOAP protokola koji je baziran na XML-u za razmjenu podataka preko HTTP protokola.

Moderni sustavi koriste HTTP API, i to najčešće REST (Representational State Transfer) API. Pod HTTP API spada bilo koji API koji koristi HTTP protokol, dok su REST API-ji podskup HTTP API-ja koji poštuju REST pravila. Pojam REST prvi put koristio je Roy Fielding u svojoj doktorskoj disertaciji 2000. godine. REST se sastoji od šest principa [2]:

- Klijent – poslužitelj arhitektura
- Komunikacija bez stanja (engl. *stateless*)
- Korištenje priručne memorije (keširanje)
- Jedinstveno sučelje – sve operacije su jedinstvene
- Slojevitost sustava

- Moguće proširenje funkcionalnosti programskim kodom

REST je i danas najpopularniji arhitektonski stil [3] za izgradnju raspodijeljenih sustava, a raširen je zbog svoje jednostavnosti, efikasnosti te prilagođenosti postojećem softveru. U sljedećem primjeru može se vidjeti nekoliko jednostavnih REST zahtjeva:

- GET /users
- POST /users
- PATCH /users/15

1.2. Dokumentacija API-ja

Da bi programer mogao poslati neki REST zahtjev, mora znati na koju krajnju točku (engl. *endpoint*) će ga poslati, a to će saznati iz dokumentacije. Dokumentacija je ključni element svakog sustava jer na strukturiran i jednostavan način programeru govori sve o tome čemu služi i što nudi API s kojim radi. Ako nema dokumentacije ili je loše napravljena, programer bi morao čitati kod ili komunicirati s autorom API-ja kako bi saznao stvarnu svrhu određene krajnje točke.

API dokumentacija sastoji se od adresa krajnjih točaka, vrsta HTTP metoda koje određena krajnja točka prima, tipova podataka, obveznih parametara, mehanizama autentifikacije i primjera zahtjeva koji se šalju te odgovora na njih. Do problema dolazi kada se nešto u sustavu promijeni, primjerice tip podataka koje prima određena krajnja točka, način autentifikacije ili jednostavno adresa krajnje točke. Tada se i dokumentacija treba promijeniti sukladno promjenama u sustavu, ali to često nije slučaj, pogotovo ako je dokumentacija pisana ručno kao neka vrsta tekstualnog dokumenta (primjerice u Microsoft Wordu). Često je slučaj da se dokumentacija i promijeni, ali bude pogrešna jer je više ljudi paralelno radilo na istom dijelu sustava. Ovo može rezultirati nekonzistentnom ili kontradiktornom dokumentacijom, što je programeru možda i gore nego da je uopće nema jer mu samo dodatno komplicira posao. Sve ovo može biti posljedica ručno pisane dokumentacije baš zato što njezin sadržaj ovisi o autoru, ali to nije jedini pristup kreiranju dokumentacije.

Takva dokumentacija zove se statična dokumentacija, jer je za svaku promjenu u sustavu potrebno ručno ažurirati dokumentaciju. Ovaj pristup upravljanja dokumentacijom je legitiman i fleksibilan jer se dokumentacija može napisati i formatirati točno onako kako je

autor to zamislio, ali ipak nije idealan pristup jer lako dolazi do pogrešaka i nekonzistentnosti između dokumentacije i samog sustava.

Kao rješenje tog problema javlja se dinamična dokumentacija. Dinamična dokumentacija je ista API dokumentacija kao i statična, samo što nema ljudskog autora, već se automatski generira bilo iz koda, ili iz specifikacije. Prednosti ovakvog upravljanja dokumentacijom očituju se u tome da se ona automatski ažurira s promjenama u sustavu pa se samim time eliminira potreba za ručnim održavanjem ispravnosti dokumentacije. Alati koji nude automatsko generiranje dokumentacije često imaju i neke dodatne mogućnosti, kao što je testiranje krajnjih točaka slanjem HTTP zahtjeva pomoću interaktivnog web-sučelja. Jedina mana ovakvog pristupa dokumentaciji je ta što se na ovaj način ne može u potpunosti urediti i organizirati dokumentacija na način na koji bi to autor htio pa dokumentacija možda neće sadržavati neke detalje poput komentara ili dijagrama koji bi bili bitni za razumijevanje poslovne logike sustava.

Prvi pokušaji dinamične dokumentacije pojavili su se početkom 2000-ih godina u vidu jezika WSDL (Web Services Description Language) kojeg su zajednički razvili IBM i Microsoft. WSDL je jezik koji je opisivao funkcionalnosti SOAP web-servisa generirajući XML datoteku s opisom svih krajnjih točaka te ostalih relevantnih informacija koje opisuju taj web-servis [4]. Kao evolucija WSDL-a razvijen je WADL (Web Application Description Language) koji je radio na sličan način kao i njegov prethodnik, samo što je opisivao REST web-servise [5].

Sa sve većom zastupljenošću REST arhitekture u suvremenim distribuiranim sustavima došlo je do potrebe za alatima koji će na precizniji, moderniji i pristupačniji način opisati i dokumentirati web-servise. Pokušaji poput WSDL-a, koji nije bio prikladan za moderne sustave jer je bio baziran na SOAP protokolu, te WADL-a koji je, iako prilagođen REST arhitekturi, bio previše kompleksan i temeljen na XML-u nisu odgovarali zahtjevima modernih web-okruženja te nisu bili široko prihvaćeni.

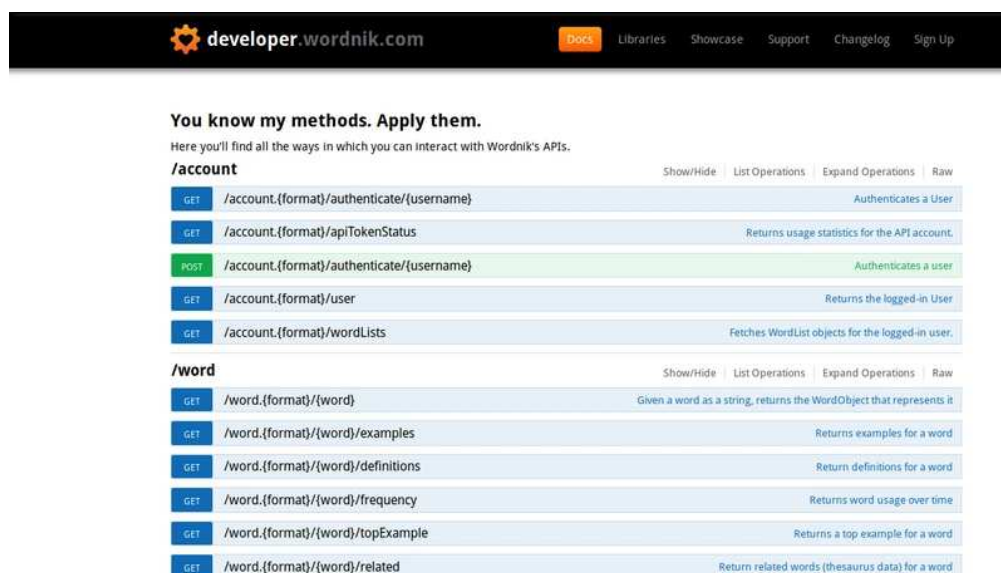
Početkom 2010.-ih godina pojavio se Swagger. Swagger je paket alata koji je namijenjen za razvoj, opisivanje, dokumentiranje i interakciju s API-jima kojeg je razvio Tony Tam kao zaposlenik američke tvrtke Wordnik [6]. Nedugo nakon, sama specifikacija za opis API-ja izdvojena je iz cjelokupnog Swagger projekta te je dobila ime OpenAPI Specification. OpenAPI je bolje prilagođen REST konceptima te koristi JSON/YAML format za opis specifikacije u usporedbi s WADL-om koji je temeljen na XML-u. Osim toga, OpenAPI je još od ranih verzija specifikacije imao kvalitetne alate za jednostavan rad sa specifikacijom

kao što su Swagger UI i Swagger Editor. Ovim alatima može se pristupiti u internetskom pregledniku bez složene konfiguracije pa je OpenAPI specifikacija vrlo brzo postala široko prihvaćen i popularan standard za razvoj i dokumentiranje API-ja.

2. OpenAPI specifikacija

2.1. Swagger

2009. godine Tony Tam, američki softverski inženjer iz San Matea u Kaliforniji, radio je u kompaniji Wordnik. Wordnik je online platforma koja je funkcionirala kao rječnik engleskog jezika koja i danas postoji. Platforma je imala REST API baziran na JSON-u koji je imao velik broj krajnjih točaka i programera koji su koristili njihov API. Kako bi lakše organizirali i dokumentirali svoj API, Tony Tam započeo je rad na sustavu koji bi im omogućio baš to, opis svake krajnje točke sa svim parametrima i ostalim relevantnim podacima koji su potrebni svakom tko koristi taj API. Ideja je bila da sustav automatski generira dokumentaciju i ažurira je pri svakoj promjeni u API-ju. Sustav je prvotno bio nazvan SWAGR te je kasnije preimenovan u Swagger čija je prva verzija Swagger Specification 1.0 objavljena 10. kolovoza 2011. godine [6]. Cilj prve verzije sustava bila je standardizacija načina opisivanja REST API-ja, a bila je jednostavna i prilagođena potrebama Wordnikove arhitekture. Swaggerovo prvo sučelje može se vidjeti u nastavku (Slika 2.1). Godinu dana kasnije, u kolovozu 2012. godine objavljena je verzija 1.1 Swagger specifikacije. Verzije 1.0 i 1.1 nisu bile široko korištene već su se uglavnom koristile interno unutar Wordnika pa se o njima i ne zna previše.



Slika 2.1 Swaggerovo sučelje s Wordnikovim API-jem

Prva verzija koja je prošla zapaženo izvan okvira Wordnika bila je verzija 1.2, objavljena u ožujku 2014. godine [7]. Swagger 1.2 uveo je razdvajanje opisa API-ja u dva dijela:

- Resource listing – korijenski dokument koji služi kao popis API resursa s njihovim URL-ovima
- API declaration – detalji krajnjih točaka za svaki API resurs

Osim toga, ova verzija je donijela prvi pokušaj usklađivanja sa standardom JSON schema draft 4. Sigurnost je bila jedan od glavnih fokusa ove verzije pa su uvedene promjene vezane za autorizaciju u vidu mehanizama kao što je OAuth2.

Swagger Specification 2.0, koji je objavljen u rujnu 2014., bio je najveći redizajn specifikacije te temelj za ono što se danas poznaje pod nazivom OpenAPI specifikacija. Resource listing i API declaration dokumenti uvedeni u prethodnoj verziji spojeni su u jedan dokument u kojem su se nalazile sve informacije koje su potrebne za precizan opis sustava. Po prvi puta je uvedena podrška za YAML format podataka pa je tako specifikacija mogla biti napisana u JSON formatu ili u YAML formatu. Uvedena je podrška za referenciranje JSON objekata, što je omogućilo recikliranje koda te je smanjilo opseg posla programerima [8].

2.2. Preimenovanje u OpenAPI

U ožujku 2015. godine, američka tvrtka SmartBear Software kupila je prava na specifikaciju te ih donirala novoj organizaciji zvanoj OpenAPI Initiative koja je bila pod pokroviteljstvom Linux fondacije, a od ostalih kompanija koje su bile osnivači inicijative izdvajaju se Google, Microsoft i IBM [9]. Ime specifikacije promijenjeno je u OpenAPI Specification, dok je tvrtka SmartBear Software zadržala vlasništvo nad Swaggerovim alatima kao što su Swagger UI i Swagger Editor. U srpnju 2017. godine objavljena je verzija specifikacije 3.0 koja je uvela promijenjenu strukturu OpenAPI dokumenta tako što je uveden objekt pod imenom *components* unutar kojeg se nalaze sheme, parametri, odgovori, primjeri i ostali podaci koji definiraju specifikaciju. Uz to, ova verzija specifikacije bila je usklađena sa standardom JSON schema draft 5. Ova verzija je danas najraširenija, iako ne i najnovija, jer je u veljači 2021. objavljena verzija 3.1, a u rujnu 2025. posljednja verzija 3.2.

2.3. Struktura OpenAPI dokumenta

OpenAPI dokument ili specifikacija glavni je dokument koji opisuje neki API, a može biti napisan u JSON ili YAML formatu. Iz ovog dokumenta generira se dokumentacija ili

poslužiteljski kod, dok se dokument istovremeno može čitati kao tekstualna datoteka pa programer može saznati sve informacije o API-ju provjerivši samo nju. U sljedećem dijelu rada predstavlja se šest glavnih sekcija svakog OpenAPI dokumenta te neki od njihovih elemenata [10]. Važno za naglasiti je da redoslijed sekcija i elemenata unutar OpenAPI dokumenta nije strogo definiran, ali će se predstaviti tipičan redoslijed koji je općeprihvaćen.

2.3.1. Opće informacije (info)

Svaki OpenAPI dokument mora sadržavati verziju OpenAPI specifikacije koja se koristi, a ona se tipično nalazi na početku svakog OpenAPI dokumenta. Nakon toga, prva sekcija koja definira API obično je sekcija *info*. Ona pruža osnovne informacije o API-ju. Sekcija *info* može sadržavati ova polja:

- `title` (naslov), obvezno polje
 - `description` (opis API-ja)
 - `version` (verzija API-ja), obvezno polje
 - `contact` (kontakt informacije) s poljima:
 - a. `name` (ime odgovorne osobe)
 - b. `url` (url API-ja)
 - c. `email` (elektronička adresa organizacije koja pruža API)
 - `license` (licenca)
 - a. `name` (naziv licence)
 - b. `url` (web-adresa licence)
- ```
openapi: 3.1.0
info:
 title: Web Shop API
 description: API za upravljanje web-trgovinom
 version: 1.0.0
 contact:
 name: Web Shop API Support
 email: support@webshop.hr
```

Kôd 2.1 – Početak OpenAPI dokumenta s verzijom i općim informacijama u YAML formatu

### 2.3.2. Poslužitelji (servers)

Sljedeća sekcija OpenAPI dokumenta je *servers*. Ona služi za definiranje URL-ova na kojima je API dostupan. Sekcija *servers* može sadržavati sljedeća polja:

- `url` (web-adresa), obvezno polje
- `description` (opis web-adrese)
- `variables` (varijable)
  - a. `default` (zadana vrijednost varijable)
  - b. `enum` (dopuštene vrijednosti varijable)
  - c. `description` (opis varijable)

Polje *variables* služi za definiranje varijabilnih vrijednosti unutar URL-a kako bi adrese mogle primiti dinamične vrijednosti, čime se smanjuje potreba za ponavljanjem sličnih URL-ova.

```
{
 "servers": [
 {
 "url": "http://localhost:8080/api/{version}",
 "description": "API server",
 "variables": {
 "version": {
 "default": "v1",
 "enum": [
 "v1",
 "v2",
 "v3"
],
 "description": "Verzija API-ja"
 }
 }
 }
]
}
```

Kôd 2.2 – Primjer *servers* sekcije OpenAPI dokumenta u JSON formatu

### 2.3.3. Grupiranja (tags)

Tipično treća sekcija OpenAPI dokumenta je *tags* i služi za definiranje tagova unutar API-ja. Tagovi služe za grupiranje krajnjih točaka prema funkcionalnosti. Sekcija *tags* je opcionalna što znači da ne mora postojati u svakom OpenAPI dokumentu, a sastoji se od sljedećih polja:

- `name` (naziv *taga*), obvezno polje

- `description` (opis *taga*)
  - `externalDocs`: (vanjska dokumentacija)
    - a. `url` (web-adresa dodatne dokumentacije)
    - b. `description` (opis dodatne dokumentacije)
- ```

tags:
  - name: Auth
    description: Autentifikacija i autorizacija
  - name: Users
    description: Upravljanje korisnicima
  - name: Products
    description: Upravljanje proizvodima
  - name: Orders
    description: Upravljanje narudžbama

```

Kôd 2.3 – Primjer *tags* sekcije OpenAPI dokumenta u YAML formatu

2.3.4. Putanje (paths)

Četvrta po redu sekcija OpenAPI dokumenta koja će se predstaviti je *paths*. Unutar *paths* sekcije definiraju se putanje do svake krajnje točke pa je ovo često i najduža sekcija nekog OpenAPI dokumenta. Svaka putanja definira krajnju točku na koju se šalje zahtjev pomoću određene HTTP metode (GET, POST, PUT, DELETE, PATCH) te sve parametre, tijelo zahtjeva i moguće odgovore koje će krajnja točka vratiti. Osim toga, svakoj putanji može se dodijeliti njezin identifikator, kratak ili detaljan opis funkcionalnosti te definirati sigurnosne zahtjeve koji određuju je li krajnja točka dostupna svima i je li za pristup potrebna autentifikacija. Sekcija *paths* obvezna je u svakom OpenAPI dokumentu. Za svaku krajnju točku definira se njezina putanja i HTTP metoda te sljedeća polja:

- `operationId` (identifikator operacije)
- `summary` (sažetak funkcionalnosti krajnje točke)
- `description` (detaljan opis funkcionalnosti krajnje točke)
- `tags` (*tag* ili grupa koje se krajnja točka tiče)
- `parameters` (parametri koje krajnja točka prima)
 - a. `$ref` (referenca na `parameter` objekt iz *components* sekcije – više o tome kasnije)
- `requestBody` (tijelo zahtjeva kojeg putanja prima)
 - a. `$ref` (referenca na `request body` objekt iz *components* sekcije)
- `responses` (odgovori poslužitelja na zahtjev)

- a. HTTP status kod
 - 1. \$ref (referenca na response objekt iz *components* sekcije)
- security (sigurnosne postavke)
 - a. naziv sigurnosne sheme (referenca na security Scheme objekt definiran u *components* sekciji)
 - b. vrijednost sigurnosne sheme
- deprecated (označavanje kao zastarjelo)

Bitno je naglasiti da se polja *parameters*, *requestBody*, *responses* i *security* mogu definirati inline, unutar *paths* sekcije, ali radi čistijeg prikaza su ovdje samo referencirani, dok su definirani unutar *components* sekcije te se primjeri definicija tih objekata mogu vidjeti u sljedećem poglavlju.

```

paths:
  /orders/{id}:
    patch:
      operationId: updateOrderStatus
      summary: Ažuriraj status narudžbe
      tags:
        - Orders
      security:
        - bearerAuth: []
      parameters:
        - $ref: '#/components/parameters/IdParam'
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UpdateOrderStatus'
      responses:
        '200':
          description: Status narudžbe ažuriran
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/OrderResponse'
        '400':
          $ref: '#/components/responses/BadRequest'
        '401':
          $ref: '#/components/responses/Unauthorized'

```

```
'404':
  $ref: '#/components/responses/NotFound'
```

Kôd 2.4 – Primjer jedne putanje unutar *paths* sekcije OpenAPI dokumenta u YAML formatu

2.3.5. Komponente (components)

Components sekcija OpenAPI dokumenta služi za definiranje struktura podataka na koje se može referencirati u drugim sekcijama korištenjem `$ref` mehanizma. Na ovaj način smanjuje se veličina OpenAPI dokumenta jer se ne mora ponovno pisati definicija strukture podataka, a sam dokument je pregledniji i organiziraniji. Ova sekcija je opcionalna, a sastoji se od različitih nepovezanih elemenata. Najvažniji elementi koji se unutar *components* sekcije mogu definirati su sheme (*schema* objekt), parametri (*parameter* objekt), tijela zahtjeva (*request body* objekt) te odgovori (*response* objekt).

Jedna definicija sheme može biti objekt te je u tom slučaju vrlo slična objektu u objektno orijentiranom programiranju jer sadrži sva svojstva koja opisuju neki entitet. Ipak, shema može biti i primitivan tip podatka. Shema se može sastojati od sljedećih elemenata:

- `type` (tip podatka, objekt ili primitivan tip)
- `properties` (svojstva objekta, samo ako je shema objekt)
svako svojstvo posebna je shema podataka koja se definira unutar druge sheme te može sadržavati elemente koje sadrži i shema najviše razine
- `required` (polja koja su obvezna, samo ako je shema objekt)
- `format` (dodatna specifikacija tipa podatka)
- `description` (opis sheme)
- `minimum/maximum` (raspon vrijednosti za numerički tip podatka)
- `minLength/maxLength` (raspon duljine za tekstualne nizove)
- `enum` (dopuštene vrijednosti)
- `default` (zadana vrijednost)
- `example` (primjer vrijednosti)
- `items` (definiranje elemenata polja)
- `$ref` (referenca na neku drugu shemu)

```
schemas:
  RegisterRequest:
    type: object
    required:
      - name
      - email
      - password
```

```

properties:
  name:
    type: string
    minLength: 2
    maxLength: 100
    example: Adrian Brnić
  email:
    type: string
    format: email
    example: adrian.brnic@fer.hr
  password:
    type: string
    format: password
    minLength: 8
    example: "lozinka123"
  birthday:
    type: string
    format: date
    example: "2002-05-26"

```

Kôd 2.5 – Primjer jedne sheme objektnog tipa definirane unutar *components* sekcije OpenAPI dokumenta u YAML formatu

Osim shema, mogu se definirati parametri, odnosno ulazne vrijednosti koje krajnje točke primaju kroz URL. Parametar može sadržavati sljedeća polja:

- `name` (naziv parametra), obvezno polje
- `in` (lokacija parametra unutar zahtjeva), obvezno polje
- `required` (označavanje parametra kao obveznog)
- `description` (opis parametra)
- `schema` (definicija sheme, opisano iznad)
- `example` (primjer vrijednosti parametra)
- `deprecated` (označavanje kao zastarjelo)

```

parameters:
  IdParam:
    name: id
    in: path
    required: true
    description: Jedinostveni identifikator resursa
    schema:
      type: integer
      format: int64

```

```
minimum: 1
```

Kôd 2.6 – Primjer jednog parametra definiranog unutar *components* sekcije OpenAPI dokumenta u YAML formatu

Tijela zahtjeva definiraju se korištenjem `request body` objekta te mogu sadržavati sljedeća polja:

- `description` (detaljan opis odgovora)
- `required` (označavanje tijela zahtjeva kao obveznog)
- `content` (sadržaj odgovora)

```
requestBodies:
  LoginRequestBody:
    description: Tijelo zahtjeva za login
    required: true
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/LoginRequest'
```

Kôd 2.7 – Primjer tijela zahtjeva definiranog unutar *components* sekcije u YAML formatu

Unutar *components* sekcije mogu se definirati i odgovori koje određena krajnja točka može vratiti korištenjem `response` objekta. Odgovori se mogu definirati i izravno unutar *paths* sekcije. Jedan odgovor može sadržavati sljedeća polja:

- `summary` (kratak opis odgovora)
- `description` (detaljan opis odgovora)
- `headers` (metapodaci odgovora)
- `content` (sadržaj odgovora)
- `links` (daljnje korištenje vraćenih podataka)

```
responses:
  RegisterResponse:
    description: Korisnik uspješno registriran
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/AuthResponse'
  BadRequest:
    description: Neispravan zahtjev
    content:
```

```

    application/json:
      schema:
        $ref: '#/components/schemas/ErrorResponse'

```

Kôd 2.8 – Primjer dvaju odgovora definiranih unutar *components* sekcije u YAML formatu

2.3.6. Sigurnost (security)

Security sekcija OpenAPI dokumenta služi za primjenu sigurnosnih shema koje se definiraju u *components* sekciji. Ova sekcija nije obvezna i često se može izostaviti ako se sigurnosne sheme primjenjuju samo na određenim krajnjim točkama API-ja. Sekcija je u usporedbi s *paths* i *components* sekcijama znatno kraća te se sastoji samo od naziva sigurnosne sheme koja je definirana u *components* sekciji unutar *securitySchemes*.

```

security:
  - bearerAuth: []

```

Kôd 2.9 – Primjer jedne *security* sekcije OpenAPI dokumenta u YAML formatu koji zahtijeva da se za svaku krajnju točku API-ja koristi *bearerAuth* sigurnosna shema

```

securitySchemes:
  bearerAuth:
    type: http
    scheme: bearer
    bearerFormat: JWT
    description: JWT token dobiven putem /auth/login
    endpointa

```

Kôd 2.10 – Primjer jedne sigurnosne sheme unutar *components* sekcije OpenAPI dokumenta u YAML formatu

U prošlom primjeru sigurnosna shema *bearerAuth* koristila bi se za svaku krajnju točku API-ja jer je tako navedeno u *security* sekciji. Ako se sigurnosna shema želi iskoristiti samo za određene krajnje točke, to se radi u *paths* sekciji OpenAPI dokumenta za svaku putanju posebno. Primjer primjene sigurnosne sheme samo na jednu krajnju točku može se vidjeti u kodu 2.4.

3. Alati za rad s OpenAPI specifikacijom

OpenAPI je „samo“ specifikacija otvorenog koda kojom upravlja OpenAPI Initiative (OAI), organizacija koja se bavi razvojem specifikacije i pravilima OpenAPI-ja. Stoga ne postoje jedinstveni ili službeni alati za korištenje specifikacije, već je svatko slobodan razviti svoje alate za uređivanje OpenAPI dokumenta, pregled dokumentacije, generiranje koda, testiranje krajnjih točaka ili bilo što drugo što mu je potrebno. Ipak, kako se OpenAPI specifikacija u svojim počecima zvala Swagger te tada nije bila izdvojena iz cijelog Swagger projekta, Swagger je nastavio postojati kao neslužbena, ali ipak najrelevantnija skupina alata za rad s OpenAPI specifikacijom. Swaggerom danas upravlja tvrtka SmartBear Software, a nudi alate kao što su Swagger Editor za pisanje specifikacije, Swagger UI za pregled dokumentacije i Swagger Codegen za generiranje koda iz specifikacije. Osim Swaggera, zajednica otvorenog koda je razvila mnoge alate koji su relevantni i korisni za rad sa specifikacijom kao što je primjerice OpenAPI Generator koji služi za generiranje klijentskog i poslužiteljskog koda.

3.1. Uređivanje koda specifikacije

3.1.1. Swagger Editor

Swagger Editor je Swaggerov proizvod koji služi za pisanje i uređivanje OpenAPI specifikacije. Jedan je od najpopularnijih izbora za pisanje specifikacije baš zbog poveznice s poviješću same specifikacije putem Swaggera. Omogućuje pisanje koda specifikacije u YAML ili JSON formatima, uvoz specifikacije te pretvorbu između YAML i JSON formata. Swagger Editor se zbog jednostavnosti najčešće koristi u internetskom pregledniku, ali se može i lokalno preuzeti na računalo za *offline* korištenje. Osim pisanja specifikacije, automatski renderira prikaz krajnjih točaka s promjenama u specifikaciji pa se odmah može pregledati dokumentacija API-ja.

3.1.2. Stoplight Studio

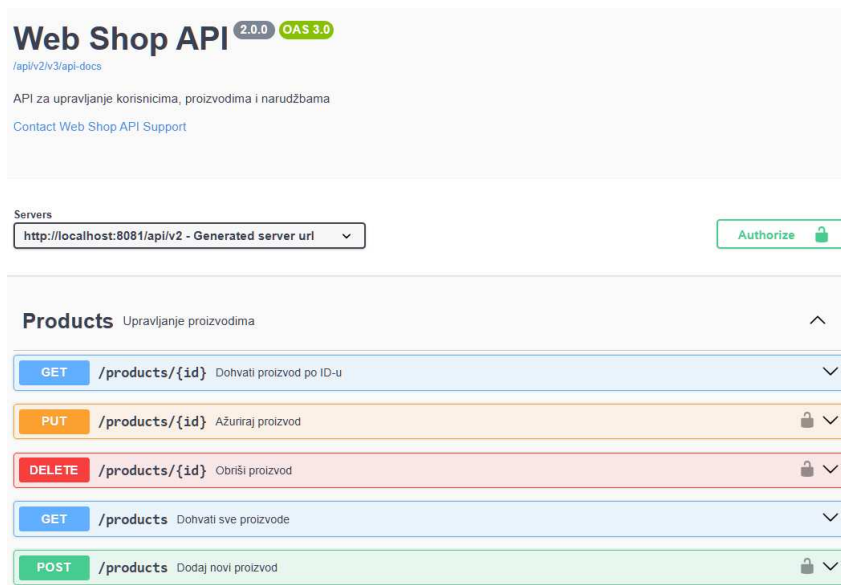
Stoplight Studio je moderan alat za pisanje i uređivanje koda OpenAPI specifikacije. Sličan je Swagger Editoru, ali nudi dodatne funkcionalnosti koje Swagger Editor nema, kao što su dizajn specifikacije putem grafičkog sučelja i integracija s Gitom [11]. Podržava YAML i

JSON format specifikacije te generiranje dokumentacije u stvarnom vremenu. Stoplight Studio naglasak stavlja na vizualnu reprezentaciju API-ja jer uklanja potrebu za ručnim pisanjem specifikacije te se često koristi za održavanje većih projekata.

3.2. Vizualizacija API-ja

3.2.1. Swagger UI

Swagger UI je alat za vizualizaciju API-ja i prikaz dokumentacije. Alat radi unutar internetskog preglednika kao web-aplikacija koja je smještena na računalu na kojem se nalazi poslužiteljski kod API-ja. Kako bi se Swagger UI-ju nekog API-ja moglo pristupiti, poslužiteljski kod API-ja mora biti pokrenut kako bi Swagger UI mogao pristupiti specifikaciji. Alat omogućuje pregled svih krajnjih točaka te njihovih HTTP metoda, traženih parametara, metoda autentifikacije i mogućih odgovora krajnje točke koji su grupirani po tagovima za bolju preglednost, kao što je prikazano (Slika 3.1). Uz to, omogućuje testiranje svake krajnje točke slanjem zahtjeva s mogućnošću mijenjanja tijela zahtjeva. Također, alat omogućava autentifikaciju korisnika tako da se, na primjer, unese vrijednost JWT tokena koji je korisniku vraćen pri uspješnoj prijavi (odnosno, pri slanju ispravne adrese e-pošte i lozinke) na krajnju točku za prijavu korisnika. Na taj način mogu se testirati neke krajnje točke koje su omogućene samo prijavljenim korisnicima. Glavna prednost ovog alata je u tome što uklanja potrebu za pisanjem klijentskog koda za testiranje API-ja ili korištenjem vanjskih alata za testiranje kao što je Postman jer se sve krajnje točke mogu testirati kroz web-sučelje Swagger UI-ja.



Slika 3.1 Sučelje Swagger UI-ja s prikazanim krajnjim točkama API-ja vezanima za proizvode web-trgovine

3.2.2. Redoc

Redoc je alat za pregled i vizualizaciju OpenAPI specifikacije otvorenog koda kojeg je razvila tvrtka Redocly. Sličan je Swagger UI-ju, ali je naglasak više stavljen na pregled dokumentacije nego interakciju jer nije moguće slati zahtjeve i testirati krajnje točke kao u Swagger UI-ju. Dizajn web-aplikacije je jednostavniji i moderniji pa je alat lakši za korištenje s API-jima koji imaju velik broj krajnjih točaka. Alat je dostupan u internetskom pregledniku bez potrebe za preuzimanjem, a koristi se kao svaka druga web-aplikacija, za razliku od Swagger UI-ja koji se najčešće pokreće lokalno. Za uvid u dokumentaciju potrebno je samo uvesti specifikaciju. Redoc ima mogućnost izvoza dokumentacije u PDF sa svim informacijama o svakoj krajnjoj točki, a osim toga, može biti korišten kao samostalna statička HTML stranica koja se može otvoriti u pregledniku.

3.3. Generiranje koda

3.3.1. Swagger Codegen

Swagger Codegen je alat za generiranje poslužiteljskog i klijentskog koda iz OpenAPI specifikacije. Alat generira početnu strukturu poslužiteljskog koda uključujući modeliranje podataka te programski kod kontrolera za pristup određenim krajnjim točkama. Radi s velikim brojem programskih jezika i radnih okvira, a najpopularniji od njih su Java Spring

Boot, C# u .NET okruženju, Python, Node.js i PHP. Osim poslužiteljskog koda, alat može generirati i klijentski kod radi jednostavnijeg pristupa API-ju od kojih se najviše koriste JavaScript i TypeScript, a također podržava i generaciju koda za mobilne aplikacije u Androidu ili Flutteru. Alat je jednostavan za korištenje putem naredbenog retka ili preko Dockera. Danas je Swagger Codegen zastarjeli alat koji se sve manje koristi, dok je sve popularniji OpenAPI Generator.

```
java -jar swagger-codegen-cli.jar generate
-i openapi.yaml
-l spring
-o spring-boot-project
```

Kôd 3.1 – Naredba za generiranje Java Spring Boot projekta iz OpenAPI specifikacije korištenjem Swagger Codegena

3.3.2. OpenAPI Generator

OpenAPI Generator je u osnovi novija verzija Swagger Codegena jer se na njemu i temelji, ali ipak modernija i prilagođena novim razvojnim okvirima. Za razliku od Swagger Codegena, OpenAPI Generator je alat otvorenog koda kojeg razvija zajednica. Zajednica je prvotno započela projekt zbog nezadovoljstva tadašnjim stanjem Swagger Codegena i nesuglasicama s tvrtkom SmartBear koja je njime upravljala zbog čega je došlo do grananja projekta (engl. *fork*). SmartBear je nastavio razvijati svoj Swagger Codegen koji je ostao povezan sa SmartBear-om i njihovom strukturom upravljanja, dok je zajednica otvorenog koda nastavila razvijati OpenAPI Generator te je prihvatila otvoreniji i decentraliziraniji pristup [12]. Oba alata imaju slične funkcionalnosti, ali OpenAPI Generator ima češće nove verzije te je bolje prilagođen za novije verzije OpenAPI specifikacije (3.0 i novije), a uz to ima više opcija konfiguracije. OpenAPI Generator je danas uglavnom bolji izbor za razvoj novog API-ja, dok se Swagger Codegen koristi za starije projekte.

```
openapi-generator-cli generate
-i openapi.yaml
-g python
-o python-client
```

Kôd 3.2 – Naredba za generiranje Python projekta iz OpenAPI specifikacije korištenjem OpenAPI Generatora

3.4. Validacija

Validacija OpenAPI specifikacije ključna je stavka u razvoju API-ja korištenjem OpenAPI specifikacije. U slučaju neispravne specifikacije došlo bi do problema s generiranjem koda ili konzistentnošću dokumentacije. OpenAPI Generator, iako nije primarno alat za validaciju, čita specifikaciju kako bi generirao kod, ali samim time i validira specifikaciju. Drugim riječima, poslužiteljski ili klijentski kod se neće moći generirati ako dana specifikacija nije validan OpenAPI dokument. Osim OpenAPI Generatora, i Swagger Editor omogućava validaciju specifikacije jer kontinuirano provjerava je li napisana specifikacija u skladu sa standardom.

3.4.1. Spectral

Spectral je alat čija je primarna svrha validacija i statička analiza (engl. *linting*) OpenAPI dokumenta. Osim validacije, alat provjerava stil i kvalitetu specifikacije, uključujući postojanost svake sheme i komponente na koju se specifikacija referencira korištenjem `$ref` mehanizma, za razliku od Swagger Editora koji neće javiti grešku ako komponenta na koju se referencira ne postoji. Alat također omogućava definiranje vlastitih pravila kao što je obveza korištenja nekih inače opcionalnih polja kako se ne bi mogla izostaviti iz specifikacije (npr. opisi ili licence).

3.5. Testiranje

Testiranje je ključan čimbenik u razvoju API-ja jer osigurava da isti radi onako kako je zamišljeno. API se može testirati korištenjem Swagger UI-ja slanjem zahtjeva na svaku krajnju točku. Ovo u praksi može biti dugotrajno te lako može dovesti do pogrešaka u testiranju, pogotovo ako sustav vrati neočekivan odgovor, a to se ne primijeti na vrijeme.

3.5.1. Dredd

Dredd je alat otvorenog koda koji služi za testiranje HTTP API-ja. Funkcionira tako da čita OpenAPI dokument te šalje HTTP zahtjeve s parametrima i tijelom zahtjeva kakvi su definirani u specifikaciji na svaku krajnju točku API-ja. Provjerava status i tijelo odgovora koje je primio od krajnjih točaka i uspoređuje ih s očekivanim odgovorima koji su navedeni u specifikaciji. Za razliku od Swagger UI-ja, ovaj pristup omogućuje provjeru svih krajnjih

točaka odjednom, a ako dođe do promjene u OpenAPI dokumentu, Dredd samo treba pročitati novu verziju dokumenta kako bi prilagodio testove novoj specifikaciji. Ipak, alat ne ulazi duboko u logiku implementacije, već testira osnovne informacije definirane u specifikaciji, pa je za testiranje rubnih slučajeva potrebno ručno pisati testove.

4. Ogledni sustav

4.1. Opis sustava

Za potrebe praktične demonstracije API-ja izrađenog pomoću OpenAPI specifikacije implementiran je sustav web-trgovine. Web-trgovina ima korisnike koji mogu naručiti proizvode te administratore koji upravljaju proizvodima i narudžbama. Korisnik postaje registrirani korisnik kada se registrira te se za svaki daljnji pristup računu mora prijaviti. Osim naručivanja proizvoda, korisnik može pratiti status narudžbe, uređivati podatke profila ili promijeniti lozinku. Administrator mijenja status narudžbi, dodaje, uređuje ili briše proizvode i njihovu količinu te upravlja korisnicima. API web-trgovine odabran je jer može demonstrirati sve glavne mogućnosti REST API-ja dokumentirane pomoću OpenAPI specifikacije kao što su slanje različitih HTTP zahtjeva, autentifikacija korisnika i pristup temeljen na ulogama (engl. *role-based access*), verzioniranje i sigurnost API-ja.

4.2. Arhitektura sustava i tehnologije

Uz opisane alate vezane za rad s OpenAPI specifikacijom, za izgradnju oglednog sustava koristile su se različite tehnologije. Sustav se temelji na troslojnoj arhitekturi, s bazom podataka te poslužiteljskim (engl. *backend*) i klijentskim (engl. *frontend*) dijelom aplikacije. Poslužiteljski dio je središnji dio API-ja te prima HTTP zahtjeve, odgovara na njih, provodi poslovnu logiku te komunicira s bazom. Klijentski dio aplikacije implementiran je u svrhu demonstracije korištenja API-ja, iako za sam API nije nužan.

4.2.1. Baza podataka

Za bazu podataka korišten je PostgreSQL. PostgreSQL je sustav za upravljanje relacijskim bazama podataka otvorenog koda. Podržava ACID (atomarnost, konzistentnost, izolacija, izdržljivost) svojstva transakcija što ga čini pouzdanim i stabilnim. Ima napredne mogućnosti indeksiranja te visoke performanse i skalabilnost što ga čini jednim od najpopularnijih izbora za upravljanje relacijskim bazama podataka.

4.2.2. Poslužiteljski dio aplikacije

Za poslužiteljski dio aplikacije korišten je razvojni okvir Spring Boot za programski jezik Java. Spring Boot je popularan izbor za razvoj poslužiteljskog dijela aplikacija utemeljenih na Javi jer omogućuje brz i jednostavan razvoj podijeljen na slojeve. Lako se integrira s različitim bazama podataka jer omogućuje objektno-relacijsko mapiranje (engl. *ORM*). Često se koristi u mikroservisima i distribuiranim sustavima. Podržava različite mehanizme za realiziranje REST API-ja te omogućuje implementaciju modernih sigurnosnih mehanizama.

Spring Boot aplikacija je organizirana po slojevima. Najniži sloj je *repository* i služi za komunikaciju s bazom podataka obavljajući CRUD operacije. Spring Boot također ima ugrađene metode za koje nije potrebno pisati SQL upit jer se njihova svrha zaključuje iz naziva metode korištenjem Spring Data JPA. Primjerice, korisnici se mogu dohvatiti po adresi elektroničke pošte definiranjem odgovarajuće metode u repozitoriju.

Srednji sloj *service* služi za provođenje poslovne logike aplikacije, primjerice za provjeru statusa pošiljke. Sloj *controller* je najviši sloj aplikacije. On služi za definiranje krajnjih točaka poslužiteljskog dijela aplikacije te prima definirane HTTP zahtjeve i vraća odgovore. Ako je zahtjev ispravan, sloj kontrolera može pozvati neku metodu servisnog sloja za obradu poslovne logike aplikacije.

4.2.3. Klijentski dio aplikacije

Za izradu klijentskog dijela aplikacije korištena je React.js, biblioteka temeljena na JavaScriptu. Biblioteka se temelji na pisanju ponovno iskoristivih komponenti i virtualnom DOM-u koji optimizira rad stvarnog DOM-a. React omogućava brz i jednostavan razvoj web-sučelja, često se koristi i za razvoj jednostraničnih (SPA) aplikacija te mobilnih aplikacija korištenjem biblioteke React Native. Za pokretanje projekta korišten je Vite, alat za brže pokretanje poslužitelja u usporedbi s tradicionalnim alatima. Klijentski dio aplikacije sastoji se od nekoliko stranica uobičajenih za web-trgovinu, kao što su registracija (Slika 4.1) i prijava korisnika, izbor proizvoda (Slika 4.2), dodavanje novih proizvoda i uređivanje košarice (Slika 4.3), praćenje narudžbi i uređivanje profila korisnika.

Registracija

Kreirajte novi račun

Ime i prezime

Email

Lozinka

Telefon

Datum rođenja

Registracija

Već imate račun? [Prijava](#)

Slika 4.1 Registracija korisnika

Proizvodi

+ Novi proizvod

NAZIV	OPIS	CIJENA	ZALIHE	
Kosilica	Kosilica za travu	€300.00	5	Dodaj u košaricu Uredi Obriši
Laptop	Laptop HP	€450.00	0	Dodaj u košaricu Uredi Obriši
Maskica za mobitel	Maskica za mobitel iPhone 14	€12.50	3	Dodaj u košaricu Uredi Obriši
Monitor	Monitor Samsung 27"	€200.00	8	Dodaj u košaricu Uredi Obriši
Štednjak	Štednjak Gorenje	€250.00	7	Dodaj u košaricu Uredi Obriši

Ukupno 5 proizvoda

Prethodna

1 / 1

Sljedeća

Slika 4.2 Izbor proizvoda

Košarica

PROIZVOD	CIJENA	KOLIČINA	UKUPNO	
Kosilica	€300.00	- 1 +	€300.00	Ukloni
Monitor	€200.00	- 2 +	€400.00	Ukloni

Pregled narudžbe

Kosilica × 1	€300.00
Monitor × 2	€400.00
Ukupno	€700.00

Dovršite narudžbu

Isprazni košaricu

Slika 4.3 Uređivanje košarice i narudžba odabranih proizvoda

4.3. Model podataka

Model podataka sustava sastoji se od tri glavna entiteta – korisnika, proizvoda te narudžbi.

4.3.1. Korisnik

Korisnik sustava je modeliran pomoću entiteta Korisnik te sadrži attribute prikazane u sljedećoj tablici (Tablica 4.1).

Tablica 4.1 Entitet Korisnik

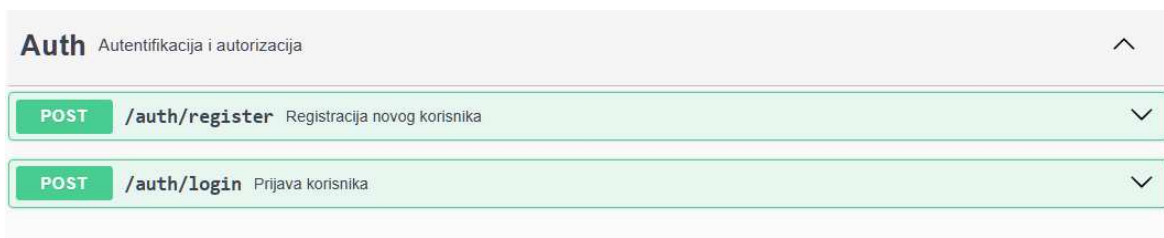
Korisnik		
Ime stupca u bazi	Tip podatka	Kratak opis
id	bigint, primarni ključ	identifikator kupca
birthday	date	datum rođenja
created_at	timestamp	vrijeme kreiranja računa
email	varchar	elektronička adresa
name	varchar	ime
password_hash	varchar	hash lozinke
phone	varchar	broj mobitela
role	varchar	uloga

Atribut `password_hash` pohranjuje kriptografski sažetak korisnikove lozinke, dok je atribut `role` prisutan samo u drugoj verziji sustava te označava je li korisnik administrator sustava ili je obični kupac. O ovome se više govori u šestom poglavlju. Prikaz krajnjih točaka s tagom *Users* unutar Swagger UI-ja može se vidjeti u nastavku (Slika 4.4).



Slika 4.4 Krajnje točke vezane za korisnike sustava

Osim krajnjih točaka s tagom *Users*, postoje još dvije krajnje točke koje se tiču korisnika, ali su grupirane s tagom *Auth*, a to su metode za registraciju i prijavu korisnika. Te metode mogu se vidjeti na sljedećoj slici (Slika 4.5).



Slika 4.5 Krajnje točke vezane za autorizaciju korisnika

4.3.2. Proizvod

Entitet Proizvod opisuje proizvod koji se može naručiti te sadrži sljedeće atribute (Tablica 4.2):

Tablica 4.2 Entitet Proizvod

Proizvod		
Ime stupca u bazi	Tip podatka	Kratak opis
<code>id</code>	bigint, primarni ključ	identifikator proizvoda

created_at	timestamp	vrijeme dodavanja proizvoda u sustav
description	varchar	opis proizvoda
name	varchar	naziv proizvoda
price	numeric	cijena proizvoda
stock	integer	dostupna količina proizvoda

Atribut `stock` predstavlja dostupnu količinu proizvoda u skladištu koja se smanjuje svakom narudžbom tog proizvoda, a samo administrator može povećati tu količinu. Krajnje točke s tagom *Products* prikazane su na sljedećoj slici (Slika 4.6).



Slika 4.6 Krajnje točke vezane za proizvode

4.3.3. Narudžba

Entitet Narudžba sastoji se od atributa prikazanih u sljedećoj tablici (Tablica 4.3):

Tablica 4.3 Entitet Narudžba

Narudžba		
Ime stupca u bazi	Tip podatka	Kratak opis
id	bigint, primarni ključ	identifikator narudžbe
created_at	timestamp	vrijeme kreiranja narudžbe
status	varchar	status narudžbe
total_price	numeric	ukupna cijena narudžbe

user_id	bigint, strani ključ	identifikator kupca
---------	----------------------	---------------------

Atribut `status` služi za praćenje statusa narudžbe te može poprimiti sljedeće vrijednosti: `CREATED` (narudžba kreirana), `PAID` (narudžba plaćena), `SHIPPED` (narudžba poslana na dostavu), `DELIVERED` (narudžba dostavljena) ili `CANCELLED` (narudžba poništena). Atribut `total_price` bilježi ukupnu cijenu narudžbe u trenutku njezinog kreiranja, dok je atribut `user_id` strani ključ koji referencira primarni ključ korisnika koji je obavio narudžbu, odnosno kupca. Krajnje točke s tagom `Orders` mogu se vidjeti ispod (Slika 4.7).

Orders Upravljanje narudžbama ^		
GET	/orders	Dohvati sve narudžbe
POST	/orders	Kreiraj novu narudžbu
GET	/orders/{id}	Dohvati narudžbu po ID-u
DELETE	/orders/{id}	Otkazi narudžbu
PATCH	/orders/{id}	Ažuriraj status narudžbe
GET	/users/{id}/orders	Dohvati narudžbe korisnika

Slika 4.7 Krajnje točke vezane za narudžbe

4.3.4. Stavka narudžbe

Između entiteta `Proizvod` i `Narudžba` postoji relacija više-na-više (N:M) te je njihov odnos modeliran entitetom `Stavka narudžbe` koji sadrži attribute prikazane u nastavku (Tablica 4.4).

Tablica 4.4 Entitet `Stavka narudžbe`

Stavka narudžbe		
Ime stupca u bazi	Tip podatka	Kratak opis
id	bigint, primarni ključ	identifikator zapisa
order_id	bigint, strani ključ	identifikator narudžbe
product_id	bigint, strani ključ	identifikator proizvoda
quantity	integer	količina proizvoda u narudžbi

unit_price	numeric	cijena u trenutku dodavanja u košaricu
------------	---------	--

Entitet Stavka narudžbe sadrži strane ključeve koji referenciraju narudžbu i proizvod te pohranjuju podatke o tome koja narudžba sadrži koji proizvod. Atribut `unit_price` sprema cijenu proizvoda u trenutku dodavanja u košaricu jer se cijena proizvoda naknadno može promijeniti.

5. Pristupi razvoju API-ja

Osnovna motivacija za razvoj onoga što danas znamo pod imenom OpenAPI specifikacija bila je dokumentiranje API-ja. Ipak, danas se OpenAPI specifikacija koristi za znatno više od samog dokumentiranja postojećih API-ja. Jedna od glavnih primjena specifikacije je razvoj API-ja od samog početka i zato danas OpenAPI specifikacija nudi dva potpuno različita pristupa razvoju API-ja. Pristup koji je prvi nastao bio je *code-first* pristup, odnosno pristup u kojem programski kod API-ja već u potpunosti ili djelomično postoji. Krajnji cilj ovakvog pristupa je iz programskog koda generirati OpenAPI dokument koji sadrži sve potrebne informacije za generiranje dokumentacije. Ovakav pristup koristio je Tony Tam, originalni autor Swaggera, koji je htio dokumentirati Wordnikov API. Suprotno *code-first* pristupu, razvijen je *specification-first* pristup. To je pristup kod kojeg je OpenAPI dokument početna točka cijelog sustava, te se iz njega pokušava generirati programski kod i dokumentacija. Oba pristupa jednako su kvalitetna te se i danas koriste, ali odabir pristupa ovisi o mnogim parametrima te je važno znati procijeniti koji pristup će biti pogodniji za arhitekturu određenog sustava.

U svrhu demonstracije različitih mogućnosti OpenAPI specifikacije napravljen je manji demonstracijski sustav u istoj domeni kao i glavni sustav opisan u prethodnom poglavlju. Za razliku od glavnog oglednog sustava koji je napravljen korištenjem *specification-first* pristupa, demonstracijski sustav je napravljen korištenjem *code-first* pristupa. Sustav se neće detaljno opisivati jer je gotovo identičan glavnom sustavu, samo je način razvoja bio drugačiji.

5.1. Pristup temeljen na kodu

Pristup temeljen na kodu, u nastavku rada *code-first* pristup, jedan je od dva osnovna pristupa u razvoju i dokumentiranju API-ja korištenjem OpenAPI specifikacije. Ovim pristupom se iz postojećeg poslužiteljskog koda generira OpenAPI dokument sa svim podacima o API-ju kojeg zbog toga ne treba ručno pisati. U ekosustavu Spring Boot-a ovaj proces omogućuje biblioteka `springdoc-openapi`. To je vanjska biblioteka koja se jednostavno integrira u postojeći Spring Boot projekt te ga analizira korištenjem Springovih anotacija. Anotacije unutar Spring Boot-a su metapodaci koji se zadaju klasama i metodama kako bi razvojnom okviru pružili dodatne informacije o svrsi određene klase ili metode. Na

primjer, anotacija `@RestController` označava da se ta klasa koristi kao kontroler, odnosno kao jedna krajnja točka sustava te ona mora imati anotaciju za relativnu putanju, primjerice `@RequestMapping("/users")`. Ova anotacija biblioteci `springdoc-openapi` signalizira da klasa koja slijedi opisuje jednu krajnju točku API-ja, u ovom slučaju skup krajnjih točaka vezanih za korisnike, te se onda ta putanja dodaje u *paths* sekciju OpenAPI dokumenta. Osim tih, koriste se još brojne anotacije, primjerice za označavanje vrste HTTP metode koja se koristi za određenu krajnju točku (`@GetMapping`, `@PostMapping`, `@DeleteMapping`, `@PutMapping`, `@PatchMapping`). Uz to, anotacije se koriste za naznačavanje vrste parametara koje određena krajnja točka prima (`@PathVariable`, `@RequestParam`). Korištenjem `@Operation` i `@ApiResponse` anotacija zadaju se kratki opis funkcionalnosti krajnje točke te mogući odgovori koje API može vratiti. Za druge razvojne okvire i programske jezike umjesto anotacija koriste se slični metapodaci kao što su dekoratori u Pythonu i atributi u .NET okruženju.

```
@PostMapping
@Operation(
    summary = "Kreiraj novog korisnika",
    responses = {
        @ApiResponse(responseCode = "201", description =
            "Korisnik uspješno kreiran"),
        @ApiResponse(responseCode = "400", description =
            "Neispravan zahtjev"),
        @ApiResponse(responseCode = "409", description =
            "Korisnik s tim emailom već postoji")
    }
)
```

Kôd 5.1 – Odlomak Spring Boot koda koji opisuje moguće odgovore na HTTP POST zahtjev s potrebnim anotacijama

Analizirajući poslužiteljski kod uz pomoć anotacija biblioteka `springdoc-openapi` dobiva potpunu sliku o svim funkcionalnostima API-ja i generira OpenAPI dokument sa svim informacijama koje ga opisuju. Biblioteka `springdoc-openapi` integrirana je sa Swagger UI alatom što omogućuje prikaz dokumentacije i testiranje krajnjih točaka kroz web-sučelje, uz pretpostavku da je pokrenut poslužiteljski kod. Osim biblioteke `springdoc-openapi`, postoje još mnoge biblioteke za druge programske jezike koje rade isti posao – analizirajući poslužiteljski kod stvaraju OpenAPI dokument.

Glavna prednost *code-first* pristupa razvoju API-ja je ta što se poslužiteljski kod može razviti na tradicionalan način. Programer nema potrebu znati ništa o samoj OpenAPI specifikaciji sve do zadnje faze razvoja sustava kada se sustav integrira s OpenAPI alatima radi generiranja dokumentacije ili lakše validacije i testiranja. Upravo tu leži ključna prednost ovog pristupa. Za primjer se može uzeti neki postojeći projekt koji je razvijen davno prije nego se i razmatrao u kontekstu OpenAPI specifikacije. Ako se specifikacija naknadno želi integrirati u projekt, cijeli proces integracije bit će iznimno jednostavan. Često se poslužiteljski kod treba samo proslijediti jednom od mnogih alata za generiranje OpenAPI specifikacije. Ipak, nekada će biti potrebno minimalno doraditi kod u svrhu dobivanja preciznije i kvalitetnije dokumentacije. Najčešće je riječ o dodavanju dodatnih anotacija koje detaljnije opisuju svrhu određenog atributa ili metode te definiraju odgovor na HTTP zahtjev kao što su gore spomenute `@Operation` ili `@ApiResponse`. Ovakav pristup pogodan je za programere koji nisu detaljno upoznati s OpenAPI specifikacijom jer im omogućuje neposredan početak razvoja poslužiteljskog koda te im pruža mogućnost za dodatno istraživanje mogućnosti specifikacije. Osim bolje prilagođenosti programerima, ovakav pristup je kvalitetan jer održava direktnu vezu između poslužiteljskog koda i dokumentacije, jer se ona generira baš iz koda. Zbog toga je praktički nemoguće uzrokovati pogreške u dokumentaciji.

Ipak, *code-first* pristup ima svoje mane. Kvaliteta dokumentacije često ovisi o tome koliko je programer detaljno opisao svaki atribut i metodu, iako će ona biti točna i konzistentna. Korištenjem ovog pristupa naglasak na dizajnu i arhitekturi sustava je znatno veći te se više resursa treba izdvojiti za razvoj jer će sustav ipak u potpunosti razviti programeri, dok se kod *specification-first* pristupa arhitektura oslanja na OpenAPI dokument te kostur koda koji je generiran iz njega. Uz to, s rastom veličine projekta sve je teže održavati i razvijati poslužiteljski kod upravo zbog toga što svaki programer anotacije piše na svoj način jer ne postoji specifikacija koje se svi trebaju pridržavati. Osim toga, generirana specifikacija često će biti manje detaljna od one koja je pisana od strane programera korištenjem *specification-first* pristupa.

5.2. Pristup temeljen na specifikaciji

Pristup temeljen na specifikaciji, odnosno *specification-first* pristup je pristup kod kojeg razvoj sustava kreće s pisanjem OpenAPI dokumenta, odnosno specifikacije. Tek nakon završetka pisanja specifikacije može se nastaviti s razvojem sustava generirajući kostur koda

korištenjem jednog od alata za generiranje poslužiteljskog koda iz OpenAPI specifikacije. Nakon generiranja kostura poslužiteljskog koda, programeri nastavljaju sa svojim uobičajenim poslom te implementiraju poslovnu logiku, povezivanje s bazom podataka, sigurnost sustava te ostale značajke koje nisu automatski generirane.

Glavna prednost *specification-first* pristupa leži u dizajnu sustava. Kada se koristi ovaj pristup, već i prije nego se napiše prva linija koda točno se zna koja krajnja točka služi čemu. Zna se koje parametre može primiti, koje odgovore može vratiti te koje sigurnosne sheme zahtijeva. Na ovaj način ne može doći do nesporazuma u razvoju poslužiteljskog koda jer je dokumentacija već gotova i prije nego je kod završen pa programeri nemaju prostora za vlastitu interpretaciju već samo implementiraju funkcionalnosti sustava. Ovakav pristup omogućava paralelan razvoj sustava što može biti iznimno važno ako API ima i korisničko sučelje. U tom slučaju tim zadužen za razvoj tog dijela sustava ne mora čekati da tim zadužen za poslužiteljski dio završi svoj posao te se ne moraju bojati da će se nešto u dizajnu sustava u zadnji tren promijeniti jer je dizajn API-ja od početka strogo definiran. Dokumentacija kreirana ovakvim pristupom uvijek je točna, konzistentna te je kvalitetna i detaljna onoliko koliko je kvalitetna i detaljna specifikacija.

Specification-first pristup zahtijeva manje ručnog pisanja koda što će često rezultirati bržim razvojem koji zahtijeva manje programera. Također, za očekivati je da će onda i broj pogrešaka u kodu biti manji što će rezultirati time da je manje vremena potrošeno na njihovo ispravljanje. Mogućnost migracije sustava iz jednog razvojnog okvira ili programskog jezika na drugi je također velika prednost *specification-first* pristupa. OpenAPI dokument se samo proslijedi generatoru koji stvori kostur poslužiteljskog koda u drugom razvojnem okviru ili programskom jeziku čime se štedi puno vremena u odnosu na uobičajenu migraciju. Osim toga, prelazak na novu verziju API-ja je puno lakši kada se sve informacije o njemu nalaze u samo jednoj datoteci.

Ovaj pristup ima i svoje nedostatke. Prvi nedostatak na koji se nailazi je nepoznavanje strukture samog OpenAPI dokumenta od strane programera te resursi potrebni da nauče više o samoj specifikaciji i načinu korištenja OpenAPI elemenata. Jednom kad nauče sve potrebno te budu spremni za početak pisanja specifikacije, pisanje same specifikacije također može biti vremenski zahtjevno. Kada je pisanje specifikacije gotovo, izbor generatora i programskog jezika koji će se koristiti za pisanje poslužiteljskog koda može imati velik utjecaj na njegovu kvalitetu budući da nisu svi alati za generiranje koda iz specifikacije jednako kvalitetni te ne generiraju jednako kvalitetan kod za sve programske

jezike. Također, svaka promjena specifikacije zahtijeva novo generiranje koda što zahtijeva prilagodbe. Ručno napisani kod koji implementira funkcionalnosti ili sigurnosne aspekte sustava ostaje prisutan u novoj generaciji, ali neke konfiguracijske datoteke kao što su `pom.xml` ili `application.properties` generator prebriše i kreira nove, što zahtijeva njihovu ponovnu konfiguraciju pri svakom novom generiranju koda. Često se može dogoditi slučaj kada kod koji je generiran nije u potpunosti prilagođen verziji razvojnog okvira. Primjerice, prilikom razvoja oglednog sustava OpenAPI Generator je generirao Java Spring Boot 2 kod s `javax` imenskim prostorom (engl. *namespace*), dok je cijeli sustav radio na verziji Spring Boot 3.2.3 koja koristi `jakarta` imenski prostor. Ovakvi problemi se ponekad mogu riješiti zadavanjem dodatnih naredbi generatoru putem konfiguracijskih zastavica, ali samo ako on ima podršku za generiranje koda za više verzija istog razvojnog okvira.

Još jedan problem koji lako može nastati nepoštivanjem pravila *specification-first* pristupa je gubitak sinkronizacije poslužiteljskog koda i OpenAPI specifikacije. Do ovakve situacije može doći ako se direktno uređuje poslužiteljski kod, odnosno dio koda kojeg je alat generirao (primjerice, nazivi krajnjih točaka). Tako bi poslužiteljski kod očekivao zahtjeve na jednoj adresi, dok će u dokumentaciji biti napisana sasvim druga adresa. Ovakav problem bi se lako detektirao i ispravio, ali ako bi se mijenjali parametri, sigurnosne sheme ili tijela zahtjeva koje prima krajnja točka, puno teže bi bilo detektirati razlog problema.

5.3. Hibridni pristup

U praksi se često dva prethodno opisana pristupa mogu koristiti zajedno. U slučaju kada se kreće sa specifikacijom kao temeljnim artefaktom sustava, prilikom implementacije poslužiteljskog koda može doći do ograničenja u dizajnu sustava. Standardnim načinom bi se ta ograničenja riješila promjenom specifikacije, ali programer ih može odlučiti riješiti izravno u kodu, bez mijenjanja specifikacije. Tek nakon što se kompletira poslužiteljski kod, iz njega se može generirati nova specifikacija. Obrnut slučaj dogodio bi se prilikom prije spomenute migracije na drugi razvojni okvir, kada bi se iz poslužiteljskog koda koji je temelj sustava kreirala specifikacija koja bi bila temelj za izgradnju novog sustava. Ipak, kod ovakvog hibridnog pristupa potrebno je uzeti u obzir sinkronizaciju poslužiteljskog koda i specifikacije jer lako može doći do nekonzistentnosti među njima.

5.4. Usporedba pristupa i kriteriji odabira

Oba prethodno opisana pristupa koriste se u mnogim projektima te su jednako relevantni i danas. Ipak, odabir pristupa koji će se koristiti za neki projekt nije slučajna već ovisi o samom projektu. Primjerice, ako je planirani API malen i sadrži samo nekoliko krajnjih točaka, često neće imati smisla trošiti puno vremena na upoznavanje programera sa specifikacijom pa je bolje izabrati *code-first* pristup. Suprotno tome, ako API ima velik broj krajnjih točaka te je kompleksan, bolji izbor bio bi uložiti više resursa na pisanje specifikacije koja će generirati kostur poslužiteljskog koda i uvelike ubrzati njegov razvoj, nego ga razvijati od nule što može biti dugotrajno. Također, ovakav pristup je bolji ako se planira da API ima više verzija ili se misli mijenjati njegov dizajn u budućnosti jer je lakše mijenjati specifikaciju nego cijeli poslužiteljski kod. *Code-first* pristup pogodniji je za timove s manje programera, dok s većim brojem ljudi koji razvijaju projekt više smisla ima *specification-first* pristup. Ako se kreira novi API, najčešće će bolji izbor biti *specification-first* pristup, dok je *code-first* pristup kvalitetniji za postojeće API-je. Nadalje, ako je API interni te služi za samo određenu aplikaciju ili sustav, *code-first* pristup može biti dobar izbor, ali ako je API javan ili je proizvod, *specification-first* je bolji izbor jer unaprijed definira svrhu i mogućnosti API-ja. Izabrani kriteriji usporedbe dvaju pristupa mogu se vidjeti u nastavku (Tablica 5.1).

Tablica 5.1 Kriteriji usporedbe pristupa

	<i>code-first</i>	<i>specification-first</i>
početak razvoja	brži, odmah pisanje koda	sporiji, dugo pisanje specifikacije
ukupno trajanje projekta	duže, ručno pisanje cijelog koda	kraće, nadopunjavanje generiranog koda
složenost projekta	manja, tradicionalno pisanje koda	veća, zahtijeva detaljno poznavanje OpenAPI specifikacije od početka
kvaliteta dokumentacije	ovisi o poslužiteljskom kodu i njegovim autorima	ovisi o specifikaciji

konzistentnost dokumentacije	uvijek ispravna jer se temelji na poslužiteljskom kodu	može doći do grešaka uzrokovanih promjenama u poslužiteljskom kodu
paralelni razvoj	moguć, ali uz dosta koordinacije među programerima	jednostavan
kvaliteta dizajna API-ja	ovisi o poslužiteljskom kodu, lako postane suboptimalna	ovisi isključivo o specifikaciji, ako dizajn nije optimalan problem će biti vidljiv u startu
verzioniranje API-ja	kompliciranije jer zahtijeva ručne promjene na poslužiteljskom kodu	jednostavnije jer je potrebno samo mijenjati specifikaciju
testiranje API-ja	nakon implementacije	prije implementacije, lakše kreiranje testova

6. Verzioniranje i sigurnost

6.1. Verzioniranje

Mogućnost jednostavnog verzioniranja API-ja jedna je od ključnih značajki OpenAPI specifikacije. S vremenom, API-ji evoluiraju te često dolazi do potrebe za mijenjanjem nekih funkcionalnosti API-ja ili uvođenja novih verzija, a najjednostavniji način za napraviti to jest promijeniti specifikaciju. Ovo poglavlje podrazumijeva korištenje *specification-first* pristupa razvoju API-ja, u suprotnom mogućnosti OpenAPI specifikacije ne dolaze do punog izražaja jer se mora izravno mijenjati programski kod.

U evoluiranju softvera postoje dvije vrste promjena. Prve su nedestruktivne promjene (engl. *non-breaking changes*). To su promjene koje samo dodaju funkcionalnosti nekog API-ja te ne mijenjaju postojeće funkcionalnosti. Drugim riječima, ako korisnik API-ja nije svjestan da su se dogodile nedestruktivne promjene, moći će ga i dalje koristiti jer se stare funkcionalnosti nisu mijenjale. Druga vrsta promjena su destruktivne promjene (engl. *breaking changes*) te one mijenjaju postojeće funkcionalnosti pa je moguće da korisnik neće moći pristupiti nekim krajnjim točkama koje su se mijenjale. Ovo poglavlje razmatra isključivo destruktivne promjene jer one zahtijevaju novu verziju API-ja.

OpenAPI specifikacija ne definira jedinstven način upravljanja verzijom API-ja, ali omogućuje korištenje različitih mogućnosti, od kojih su najčešće korištene upravljanje verzijama pomoću URL-a, upravljanje verzijama pomoću parametra zahtjeva te upravljanje verzijama pomoću HTTP zaglavlja (engl. *header*) [13].

6.1.1. Upravljanje verzijama pomoću URL-a

Ovaj način verzioniranja API-ja najrašireniji je te uglavnom najpraktičniji. Često je korišten zbog svoje jednostavnosti za implementaciju i korištenje. Kod ovog pristupa u svakom URL-u mora se navesti kojoj verziji API-ja se pristupa. Verzija API-ja može biti u osnovnom URL-u (engl. *base URL*), a može se i dodati na kraju putanje. Ovakav način verzioniranja implementira se na razini aplikacije te svaka krajnja točka sadrži odgovarajuću verziju API-ja u svom URL-u. Korištenjem ovog načina verzioniranja svaka verzija API-ja ima svoj OpenAPI dokument. Sljedeće URL adrese opisuju različite načine pristupanja različitim verzijama istog API-ja.

`http://localhost:8080/api/users`

Kôd 6.1 – Primjer URL-a za pristup krajnjoj točki API-ja prije uvođenja različitih verzija

`http://localhost:8080/api/v1/users`

Kôd 6.2 – Primjer URL-a za pristup krajnjoj točki prve verzije API-ja, koja je navedena u osnovnom URL-u

`http://localhost:8080/api/v2/users`

Kôd 6.3 – Primjer URL-a za pristup krajnjoj točki druge verzije API-ja, također navedena u osnovnom URL-u

`http://localhost:8080/api/users/v1`

Kôd 6.4 – Primjer URL-a za pristup krajnjoj točki prve verzije API-ja, ali je verzija navedena kao parametar na kraju URL-a

U sklopu oglednog sustava razvijene su dvije verzije API-ja radi demonstriranja verzioniranja korištenjem OpenAPI specifikacije. Glavna razlika između prve i druge verzije API-ja je uvođenje pristupa temeljenog na ulogama (engl. *role-based access*) u drugoj verziji gdje korisnik može biti običan kupac ili administrator. Također je promijenjena logika zahtjeva za kupnju gdje je identifikator korisnika uklonjen iz tijela zahtjeva za kupnju tako da korisnik može izvršiti kupnju samo u svoje ime. Osim toga, promijenila se logika nekih krajnjih točaka kao što je dohvat narudžbi kako bi se sustav bolje prilagodio uvođenju pristupa temeljenog na ulogama. Na taj način ove promjene postale su destruktivne jer mijenjaju tijela zahtjeva za kupnju proizvoda i prikaz narudžbi te je nastala nova verzija API-ja. Osim toga, dodane su neke nove funkcionalnosti, kao što je zahtjev za promjenu lozinke.

Proces kreiranja nove verzije API-ja započeo je promjenom OpenAPI dokumenta, u kojem su uređene sheme koje opisuju ulogu korisnika u sustavu te shema za promjenu lozinke. Promijenjena je verzija API-ja te su dodane nove krajnje točke za gore navedene funkcionalnosti. Nakon toga, u poslužiteljskom kodu su odrađene prilagodbe kako bi odgovarao specifikaciji te je tako nastala nova verzija API-ja.

6.1.2. Upravljanje verzijama pomoću parametra zahtjeva

Drugi način verzioniranja API-ja koristi parametar zahtjeva kao verziju API-ja kojoj se pristupa. Ovaj način je funkcionalan, iako ne prati dobru praksu jer se verzija API-ja navodi kao parametar zahtjeva, što ona nije. Ovaj način verzioniranja ne implementira se na razini aplikacije već se verzija API-ja dohvaća kao parametar iz svakog zahtjeva.

```
http://localhost:8080/api/users?version=1
```

Kôd 6.5 – Primjer URL-a za pristup krajnjoj točki prve verzije API-ja koja je navedena kao parametar zahtjeva

```
/users:
  get:
    parameters:
      - name: version
        in: query
        required: true
        schema:
          type: string
          enum:
            - "1"
            - "2"
```

Kôd 6.6 – Primjer odlomka OpenAPI specifikacije koji opisuje krajnju točku koja zahtijeva verziju API-ja kao parametar

6.1.3. Upravljanje verzijama pomoću HTTP zaglavlja

Ovaj način verzioniranja radi korištenjem HTTP zaglavlja. U njemu se predaje verzija API-ja što eliminira potrebu za verzijom u URL-u. Implementacija ovog načina verzioniranja složenija je od prethodna dva načina jer zahtijeva čitanje HTTP zaglavlja te zato nije široko korištena. Osim toga, testiranje API-ja postaje teže jer se verzija ne može jednostavno zadati kroz URL ili kao parametar već se mora koristiti zaglavlje HTTP zahtjeva.

6.2. Sigurnost

Sigurnost je jedan od najvažnijih aspekata svakog softvera, stoga joj se posvećuje posebna pažnja i pri radu s OpenAPI specifikacijom. OpenAPI specifikacija omogućuje precizno definiranje sigurnosnih mehanizama unutar OpenAPI dokumenta koji se mogu koristiti globalno, odnosno za cijeli sustav ili samo za određene krajnje točke ili dijelove sustava. Sigurnosne sheme se unutar OpenAPI dokumenta definiraju unutar sekcije *components*, pod *securitySchemas*, kao što je opisano u poglavlju 2.3. Važno je naglasiti da OpenAPI specifikacija ni na koji način ne prisiljava programera da koristi određeni sigurnosni mehanizam, već samo dokumentira koji mehanizam bi se trebao koristiti, dok implementacija ovisi o programeru.

OpenAPI specifikacija nudi pet različitih sigurnosnih mehanizama koji se mogu koristiti. To su HTTP autentifikacija, autentifikacija putem API ključa, OAuth2, OpenIDConnect te uzajamna TLS (engl. mutual TLS) autentifikacija [14].

6.2.1. HTTP autentifikacija

HTTP autentifikacija koristi HTTP protokol za autentifikaciju korisnika nekog sustava. Podaci se razmjenjuju unutar *Authorization* zaglavlja HTTP zahtjeva. Postoje dvije vrste HTTP autentifikacije, *Basic* i *Bearer*. *Basic* autentifikacija šalje korisničko ime te lozinku kodirane pomoću base64 algoritma, dok *Bearer* autentifikacija koristi token. Token je niz znakova koje poslužitelj šalje korisniku kao potvrdu autentifikacije, te ga korisnik može koristiti za autentifikaciju umjesto da svaki put nanovo unosi korisničko ime i lozinku do njegovog isteka. Danas najčešće korišten token u sklopu *Bearer* autentifikacije je JSON Web Token (JWT) te je on korišten i u oglednom sustavu jer je najbolje prilagođen za rad s jednostavnim REST API-jima. JWT je standard prijenosa podataka između klijenta i poslužitelja, a sastoji se od zaglavlja, tijela i potpisa. U tijelu se nalaze podaci potrebni za autentifikaciju, a u potpisu digitalni potpis tih podataka sa zaglavljem potpisani SHA-512 algoritmom. Primjer definicije HTTP autentifikacije korištenjem Bearer JWT tokena unutar OpenAPI dokumenta može se vidjeti u kodu 2.10 [15][16].

6.2.2. Autentifikacija putem API ključa

Autentifikacija putem API ključa također je podržana u sklopu OpenAPI specifikacije. Ovaj način autentifikacije funkcionira tako da se korisniku dodijeli API ključ, odnosno nasumičan niz znakova bez roka trajanja. Ovaj način autentifikacije lošiji je od autentifikacije korištenjem HTTP protokola jer se API ključ, za razliku od tokena nikad ne mijenja te ga može koristiti bilo tko ako uspije doći do njega.

6.2.3. OAuth2

OAuth2 je protokol za autorizaciju bez dijeljenja korisničkih podataka kao što su lozinke. Glavni cilj ovog protokola je dijeljenje podataka između dvaju nepovezanih sustava bez kompromitiranja osjetljivih podataka. Funkcionira tako da korisnik izabere neki vanjski servis za prijavu (primjerice Google ili Microsoft) te se preko njega odradi autorizacija, odnosno dopuštenje za pristup nekom resursu, ali ne i autentifikacija. Vanjski servis izdaje token koji se prenosi putem HTTP protokola i vrijedi određeno vrijeme. Na taj način se

korisnička imena i lozinke ne moraju pohranjivati za svaki sustav, već se koristi pouzdani vanjski servis. OAuth2 protokol koristi tokove (engl. *flows*) koji omogućuju autorizaciju ovisno o scenariju, primjerice pokušava li korisnik dobiti pristup nekom resursu ili dva servisa komuniciraju bez korisnika. Unutar OpenAPI dokumenta OAuth2 protokol se definira pomoću sigurnosne sheme tipa `oauth2` unutar *components* sekcije OpenAPI dokumenta [17][18].

6.2.4. OpenID Connect

OpenID Connect (OIDC) je protokol koji se oslanja na OAuth2, ali osim autorizacije omogućuje i autentifikaciju korisnika. Osim tokena za pristup nekom resursu, OIDC vraća token koji sadrži podatke o identitetu korisnika kako bi sustav znao tko pristupa resursima, informaciju koju protokol OAuth2 ne pruža. Zato se ova dva protokola najčešće koriste zajedno. Korištenjem OIDC protokola može se omogućiti jedinstvena prijava (SSO, engl. *single sign-on*), što omogućuje korisniku da koristi više servisa od istog pružatelja identiteta (na primjer Gmail i Google karte) bez potrebe da se prijavljuje na svakog od njih. Za definiranje OIDC protokola unutar OpenAPI dokumenta koristi se sigurnosna shema tipa `openIdConnect`. Za razliku od HTTP autentifikacije i autentifikacije putem API ključa koje se koriste za manje API-je, OAuth2 i OIDC se najčešće koriste za složenije sustave s velikim brojem korisnika [19].

6.2.5. Uzajamni TLS

Posljednja sigurnosna shema koju podržava OpenAPI specifikacija je autentifikacija korištenjem uzajamnog ili međusobnog TLS protokola (engl. *mutual TLS*). TLS (Transport Layer Security) je protokol koji osigurava komunikaciju preko mreže, a njegova najpoznatija primjena je osiguravanje HTTP protokola u sklopu HTTPS (HTTP over TLS) protokola. Standardni TLS protokol provjerava identitet samo jedne strane, najčešće poslužiteljske, kako bi se klijent uvjerio da je poslužitelj stvarno onaj kojim se predstavlja. Kod uzajamnog TLS protokola obje strane imaju certifikat pa jedna drugu provjeravaju kako bi utvrdili međusobni identitet. Ovakav način autentifikacije najčešće koriste velike korporacije koje rade s iznimno osjetljivim podacima kao što su banke ili državna tijela. U OpenAPI specifikaciji uzajamni TLS se definira kao tip sigurnosne sheme `mutualTLS` [20][21].

```
components:
  securitySchemes:
```

MutualTLS:

type: mutualTLS

description: Mutual TLS authentication

Kôd 6.7 – Definicija *mutualTLS* autentifikacije u OpenAPI dokumentu

Zaključak

Upravljanje dokumentacijom jedan je od ključnih čimbenika u razvoju svakog HTTP API-ja. Uz klasičnu statičnu dokumentaciju koju u potpunosti piše i uređuje čovjek, postoje alati koji automatski generiraju dokumentaciju iz programskog koda ili specifikacije. Takva vrsta dokumentacije zove se dinamična dokumentacija. Dinamična dokumentacija uglavnom je bolji izbor jer smanjuje rizik od netočne dokumentacije, a zahtijeva manje vremena i resursa za održavanje.

Najpoznatiji standard za generiranje dinamične dokumentacije je OpenAPI specifikacija. Središnji element svakog OpenAPI projekta je OpenAPI specifikacija ili dokument. Unutar tog dokumenta je u potpunosti definiran API, sa svim krajnjim točkama, tijelima zahtjeva, mogućim odgovorima i sigurnosnim shemama koje se koriste. Za rad s OpenAPI specifikacijom postoje mnogi kvalitetni alati koji omogućuju uređivanje koda specifikacije, vizualizaciju API-ja, generiranje koda iz specifikacije te testiranje i validaciju, pri čemu su mnogi od njih otvorenog koda. OpenAPI specifikacija omogućuje dva pristupa razvoju API-ja: pristup temeljen na kodu te pristup temeljen na specifikaciji. Kod pristupa temeljenog na kodu, specifikacija i dokumentacija se generiraju iz programskog koda API-ja, dok se kod pristupa temeljenog na specifikaciji prvo piše specifikacija, nakon čega se iz nje generira kostur poslužiteljskog koda API-ja. U sklopu rada razvijen je ogledni sustav u svrhu demonstracije mogućnosti OpenAPI specifikacije, a u razvoju su korištena oba pristupa.

Zaključno, za ovakav tip i veličinu oglednog sustava pristup temeljen na kodu ipak se pokazao kao prikladniji, zbog velikog vremena koje je potrebno potrošiti za pisanje specifikacije, što uključuje istraživanje i ovladavanje njezinim mogućnostima. Ipak, nakon što je taj korak svladan, pristup temeljen na specifikaciji postaje intuitivniji i pristupačniji za daljnji rad, stoga je pri odabiru pristupa ključno uzeti u obzir veličinu i trajanje projekta, broj programera te dugoročne planove razvoja API-ja. Rezultati pokazuju da je pristup temeljen na kodu prikladniji za manje projekte s malo programera te omogućuje brži početak razvoja API-ja, dok prednosti pristupa temeljenog na specifikaciji dolaze do izražaja u kompleksnijim projektima s više programera koji zahtijevaju jasno definiranu arhitekturu i dobru koordinaciju među njima.

Literatura

- [1] Bosnić, I., Čavrak, I., *Otvoreno računarstvo 2022/23*, Poveznica: fer.unizg.hr/_download/repository/OR_2025_26_08a_REST_v1.pdf; pristupljeno 13. svibnja 2026.
- [2] Gupta, L., *REST Architectural Constraints*, (2024, studeni), Poveznica: <https://restfulapi.net/rest-architectural-constraints/>; pristupljeno 13. svibnja 2026.
- [3] Simpson, J., *The Top API Architectural Styles of 2025*, (2025, srpanj), Poveznica: <https://nordicapis.com/the-top-api-architectural-styles-of-2025/>; pristupljeno 13. svibnja 2026.
- [4] Christensen, E., Curbera, F., Meredith, G., Weerawarana, S., *Web Services Description Language (WSDL) 1.1*, (2001, ožujak), Poveznica: <https://www.w3.org/TR/wsdl.html>; pristupljeno 15. svibnja 2026.
- [5] Hadley, M., *Web Application Description Language*, (2009, kolovoz), Poveznica: <https://www.w3.org/submissions/wadl/>; pristupljeno 15. svibnja 2026.
- [6] Ralphson, M., *A brief history of the OpenAPI Specification*, (2018, prosinac), Poveznica: <https://dev.to/mikeralphson/a-brief-history-of-the-openapi-specification-3g27>; pristupljeno 21. svibnja 2026.
- [7] OpenAPI Initiative (OAI), *Swagger RESTful API Documentation Specification*, (2015, lipanj), Poveznica: <https://github.com/OAI/OpenAPI-Specification/blob/main/versions/1.2.md>; pristupljeno 25. svibnja 2026.
- [8] OpenAPI Initiative (OAI), *OpenAPI specification*, Version 2.0, (2014, rujan), Poveznica: <https://swagger.io/specification/v2/>; pristupljeno 26. svibnja 2026.
- [9] The Linux Foundation, *New Collaborative Project to Extend Swagger Specification for Building Connected Applications and Services*, (2015, studeni), Poveznica: <https://www.linuxfoundation.org/press/press-release/new-collaborative-project-to-extend-swagger-specification-for-building-connected-applications-and-services>; pristupljeno 26. svibnja 2026.
- [10] OpenAPI Initiative (OAI), *OpenAPI specification v3.2.0*, (2025, rujan), Poveznica: <https://spec.openapis.org/oas/latest.html>; pristupljeno 26. svibnja 2026.
- [11] Stoplight, *Stoplight Studio*, (2022, rujan), Poveznica: <https://github.com/stoplightio/studio>; pristupljeno 1. lipnja 2026.
- [12] Lowe, H., *Swagger-Codegen vs OpenAPI Generator: Which Is More Active?*, (2025, prosinac), Poveznica: <https://loquisoft.com/blog/swagger-codegen-vs-openapi-generator-which-is-more-active/>; pristupljeno 2. lipnja 2026.
- [13] Postman, *What is API versioning?*, Poveznica: <https://www.postman.com/api-platform/api-versioning/>; 8. lipnja 2026.
- [14] OpenAPI Initiative (OAI), *Describing API security*, Poveznica: <https://learn.openapis.org/specification/security.html>; pristupljeno 8. lipnja 2026.
- [15] Mozilla Developer Network, *HTTP Authentication*, Poveznica: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Authentication>; pristupljeno 10. lipnja 2026.

- [16] JWT.io, *Introduction to JSON Web Tokens*, Poveznica: <https://www.jwt.io/introduction#what-is-json-web-token>; pristupljeno 10. lipnja 2026.
- [17] Microsoft, *What is OAuth?*, Poveznica: <https://www.microsoft.com/hr-hr/security/business/security-101/what-is-oauth>; pristupljeno 11. lipnja 2026.
- [18] OpenAPI Initiative (OAI), *OAuth 2.0*, Poveznica: https://swagger.io/docs/specification/v3_0/authentication/oauth2/; pristupljeno 11. lipnja 2026.
- [19] Microsoft, *What is OIDC?*, Poveznica: <https://www.microsoft.com/en-us/security/business/security-101/what-is-openid-connect-oidc?msocid=2e21cb438ed669922334ded58fcb683a>; pristupljeno 11. lipnja 2026.
- [20] Cloudflare, *What is mutual TLS (mTLS)?*, Poveznica: <https://www.cloudflare.com/learning/access-management/what-is-mutual-tls/>; pristupljeno 16. lipnja 2026.
- [21] Speakeasy, *The Mutual TLS security scheme in OpenAPI*, Poveznica: <https://www.speakeasy.com/openapi/security/security-schemes/security-mutualtls>; pristupljeno 16. lipnja 2026.

Izjava o korištenju umjetne inteligencije

U izradi ovog rada korištena je umjetna inteligencija. Korišteni su modeli ChatGPT 5.4 (tvrtka OpenAI) te Claude Sonnet 4.6 (tvrtka Anthropic). Umjetna inteligencija korištena je za početno upoznavanje s OpenAPI specifikacijom te razumijevanje koncepata i pomoć u implementaciji oglednog sustava u vidu sintakse te ispravljanja grešaka. UI je također korištena za pomoć u radu s OpenAPI Generatorom. Nakon pisanja rada korištena je umjetna inteligencija radi uočavanja pravopisnih i gramatičkih grešaka.

Osvrt na korištenje umjetne inteligencije: Umjetna inteligencija se pokazala korisnom u svrhu bržeg upoznavanja s konceptima i mogućnostima OpenAPI specifikacije te lakšeg rada s alatima s kojim se nisam susretao kao što je OpenAPI Generator. Omogućila je brži razvoj oglednog sustava omogućivši brže rješavanje tekućih problema. Nisu uočene halucinacije.

Izjava o korištenju umjetne inteligencije: Potvrđujem da sam tijekom izrade ovog završnog rada koristio umjetnu inteligenciju u skladu s Politikom primjerenog korištenja umjetne inteligencije na Fakultetu elektrotehnike i računarstva, te da umjetna inteligencija nije zamijenila moje kritičko razmišljanje niti moj doprinos.